University of Moratuwa

EN3150 - Pattern Recognition

Assignment 03

Resource-Constrained CNN for Edge Image Classification

Group: Draco

Dealing with AI from the core

GUNARATHNA K.T.M.B.	230203G
JAYAWARDANA K.P.P.	230294H
JITHMAL P.G.I.	230306B
NIMADITH W.N.A.C.	230442L

GitHub repository:
`https://github.com/NavigatorPrime/CNN-Image-Classifier`

Source branch audited: `Final`
Source commit: `3e335db30d025e03ee89ebe7233fa23973e065f1`
Corrected working branch: `EN3150-A03-Final-Corrected-Report-v2`

October 2026

Contents

1 Introduction **2**

2 Task 1: Data Preparation **2**

2.1 Dataset and split 2

2.2 Preprocessing and reproducibility 3

3 Task 2: Custom Architecture Design **3**

3.1 Model A: Standard CNN 3

3.2 Model B: Lightweight depthwise-separable CNN 3

4 Task 3: Optimizer Selection and Tuning **4**

5 Task 4: Custom Model Training and Evaluation **5**

6 Task 5: Lightweight SOTA Fine-Tuning and Evaluation **6**

7 Task 6: Final Comparison and Trade-off Discussion **8**

7.1 Accuracy 8

7.2 Memory footprint 8

7.3 Computational cost 9

7.4 Accuracy-efficiency frontier 10

7.5 Advantages and limitations of Model B 10

7.6 Advantages and limitations of pre-trained SOTA models 10

7.7 Deployment recommendations 11

8 Overall Conclusion **11**

1 Introduction

This assignment investigates image classification under strict edge-computing constraints. The work uses the UCI DeFungi microscopic image dataset and compares two custom convolutional neural networks with two lightweight pre-trained architectures. The central engineering question is not accuracy alone, but how much accuracy is obtained for a given parameter count, checkpoint size, and arithmetic cost.

All final numerical values in this report are taken from the corrected Stage-2 artifacts. Existing Model A, Model B, MobileNetV2, and ShuffleNetV2 checkpoints were preserved; only the missing controlled optimizer-comparison experiment was newly trained. Computational cost is reported using one common convention: Conv2d and Linear multiply-accumulate operations (MACs) for a $3 \times 64 \times 64$ input. Batch normalization, activation, pooling, residual additions, concatenation, channel shuffle, softmax, and data movement are excluded. FLOPs are reported as $2\times$ MACs. These are calculated operation counts, not latency measurements.

2 Task 1: Data Preparation

2.1 Dataset and split

The selected dataset is **DeFungi** from the UCI Machine Learning Repository. It contains five fungal classes: H1, H2, H3, H5, and H6. All packaged images are RGB and exactly 64×64 pixels, satisfying the assignment's maximum-resolution requirement.

The packaged split contains 9,114 images:

Split	H1	H2	H3	H5	H6	Total
Train	3,082	1,633	573	572	517	6,377
Validation	660	350	122	122	110	1,364
Test	662	351	124	124	112	1,373

This corresponds to approximately 69.97%, 14.97%, and 15.06%, with the small deviation from exact percentages caused by integer class-wise splitting. The same packaged train/validation/test directories are used by all four models.

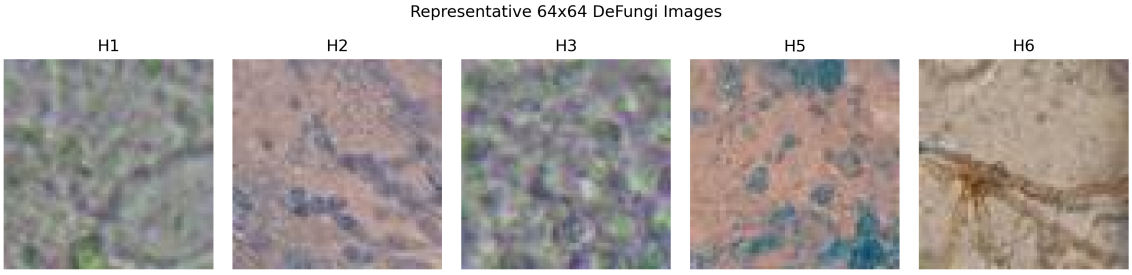


Figure 1: Representative downscaled 64×64 samples from the five DeFungi classes.

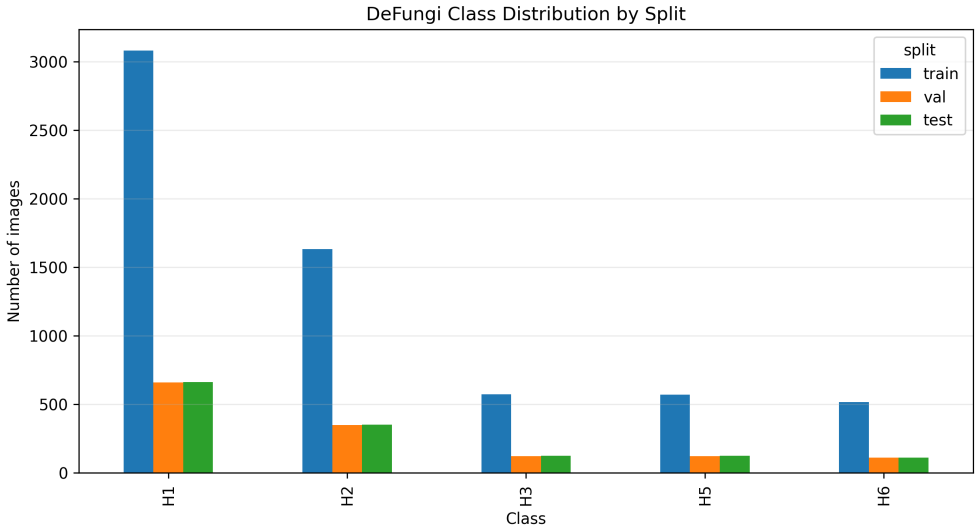


Figure 2: Class distribution across the common train, validation, and test split. The dataset is imbalanced, with H1 and H2 much more frequent than H3, H5, and H6.

2.2 Preprocessing and reproducibility

Training images use resize-to-64×64, random horizontal and vertical flips, random rotation of $\pm 15°$, color jitter, conversion to tensors, and ImageNet normalization. Validation and test images use deterministic resize, tensor conversion, and the same normalization without augmentation. The corrected code resolves paths relative to the repository, uses the existing split manifest, and applies an explicit seed for reproducible future runs.

3 Task 2: Custom Architecture Design

3.1 Model A: Standard CNN

Model A interleaves standard 3×3 convolutions with BatchNorm, ReLU, and 2×2 max-pooling. Channel width grows from 32 to 256 while spatial resolution is reduced from 64×64 to 4×4. The classifier is $4096 \to 512 \to 128 \to 5$ with dropout.

The exact trainable parameter count is **2,553,349**. The corrected calculation is:

$$\begin{aligned} \text{feature extractor} &= 389,376, \\ \text{classifier} &= 2,163,973, \\ \text{total} &= 2,553,349. \end{aligned}$$

ReLU is hardware-friendly because it is piecewise linear and avoids transcendental operations such as exponentials. This is advantageous on resource-constrained processors and common integer inference kernels.

3.2 Model B: Lightweight depthwise-separable CNN

Model B contains a small standard stem followed by six depthwise-separable convolution blocks, Global Average Pooling, dropout, and a $128 \to 5$ classifier. Each block factorizes a standard convolution into a depthwise 3×3 operation and a pointwise 1×1 projection.

For a standard convolution with M input channels, N output channels, kernel size D_K, and output spatial size $D_G \times D_G$:

$$\text{MACs}_{\text{std}} = D_K^2 M N D_G^2. \tag{1}$$

For depthwise separable convolution:

$$\text{MACs}_{\text{sep}} = D_K^2 M D_G^2 + M N D_G^2. \tag{2}$$

The ratio is

$$\frac{\text{MACs}_{\text{sep}}}{\text{MACs}_{\text{std}}} = \frac{1}{N} + \frac{1}{D_K^2}. \tag{3}$$

For 3×3 kernels, this yields a substantial per-block reduction.

The exact trainable parameter count of Model B is **37,957**, which is 62.04% below the assignment ceiling of 100,000 parameters. Relative to Model A, Model B uses **98.51% fewer parameters**, or approximately **67.27× fewer**. ReLU6 is used as a bounded piecewise-linear activation; its bounded output range can be useful for quantization, while still avoiding expensive transcendental evaluation.

4 Task 3: Optimizer Selection and Tuning

A controlled comparison was run on Model B for 20 epochs using identical initial weights, the same data split, seed 42, batch size 32, cross-entropy loss, augmentation, weight decay (10^{-4}), and validation-accuracy checkpoint selection. The compared optimizers were standard SGD, SGD with momentum, and Adam.

Table 1: Controlled optimizer comparison on Model B.

Optimizer	LR	Momentum	Best Val. Acc.	Best Epoch	Test Acc.	Macro F1
Standard SGD	0.010	0.0	67.01%	19	65.91%	0.5770
SGD + Momentum	0.010	0.9	70.09%	12	67.23%	0.6356
Adam (selected)	0.001	–	**72.29%**	18	**70.14%**	**0.6737**

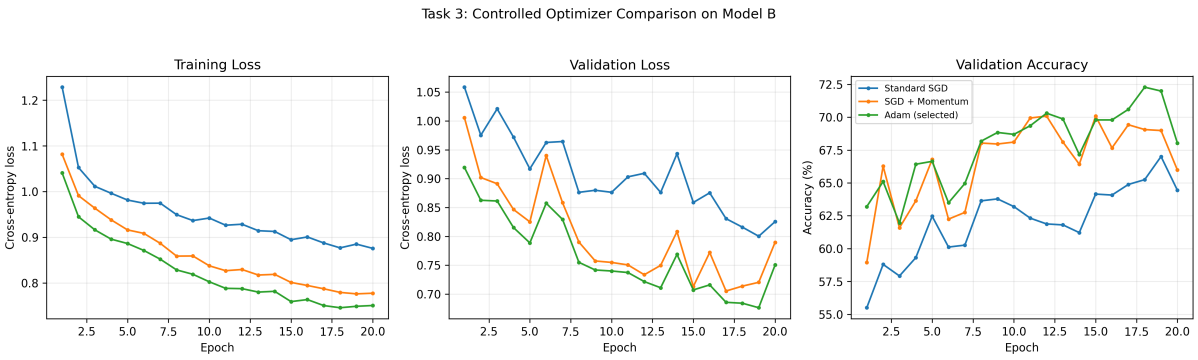


Figure 3: Controlled Task-3 optimizer comparison. Momentum accelerates SGD convergence and improves the final metrics, while Adam provides the strongest overall validation and test performance in this experiment.

Momentum updates the running direction using a velocity term. A common form is

$$v_t = \mu v_{t-1} + g_t,$$
$$\theta_{t+1} = \theta_t - \eta v_t,$$

where μ is the momentum coefficient. In the experiment, SGD with momentum reached 66.28% validation accuracy by epoch 2 and peaked at 70.09% by epoch 12, while standard SGD reached a lower 67.01% peak only at epoch 19. Thus, momentum provided faster convergence and better final performance under the controlled setup. Adam was selected because it obtained the best validation accuracy, test accuracy, and macro F1.

5 Task 4: Custom Model Training and Evaluation

Both custom networks were trained for 20 epochs, meeting the assignment requirement. Their main Task-4 runs are independent of the controlled Task-3 optimizer experiment.

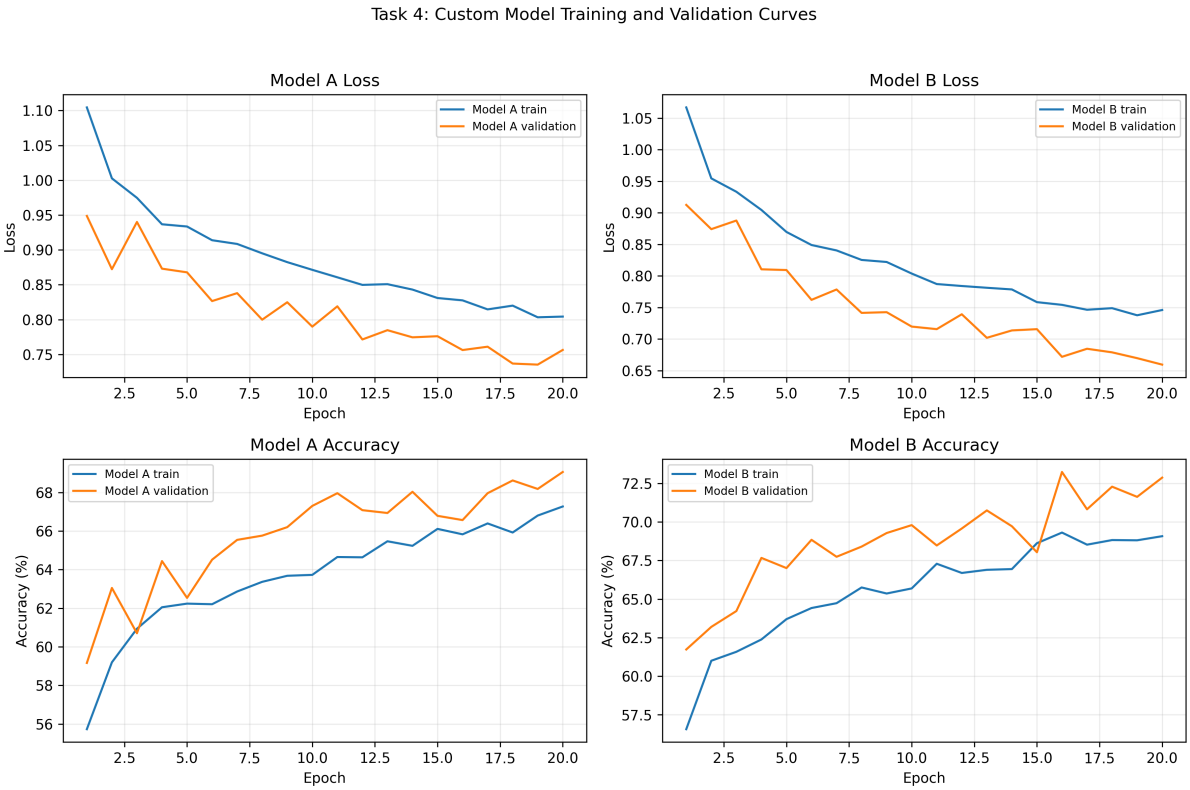
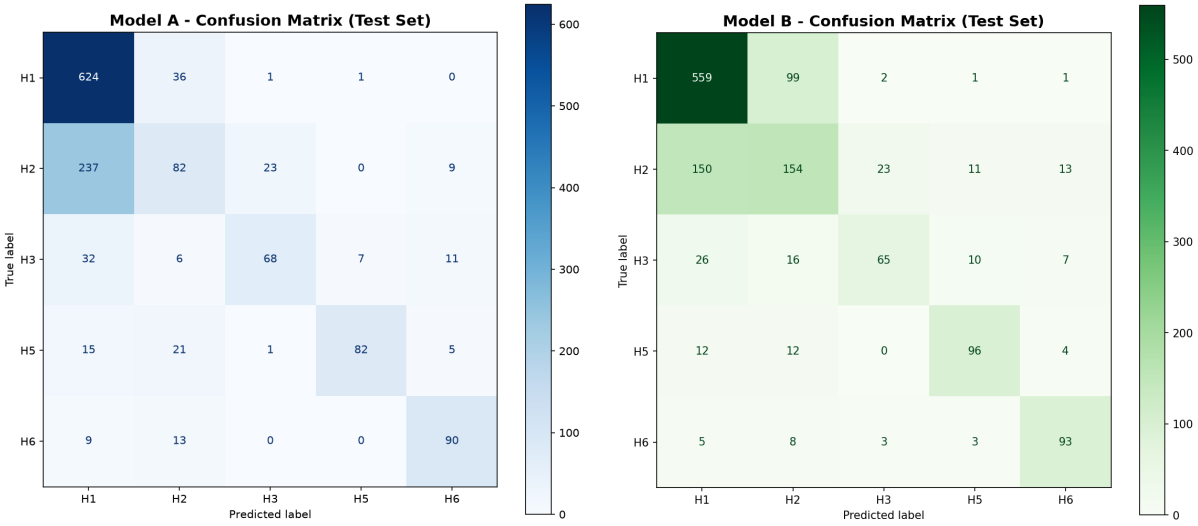


Figure 4: Training and validation curves for Model A and Model B over the 20-epoch custom-model experiments.



(a) Model A. **(b)** Model B.

Figure 5: Test-set confusion matrices for the two custom networks.

Table 2: Model A vs Model B comparison. Checkpoint sizes are actual PyTorch file sizes.

Metric	Model A	Model B
Trainable parameters	2,553,349	37,957
Checkpoint size	9.755 MiB	180.16 KiB
FP32 parameter storage	9.740 MiB	148.27 KiB
Avg. training time / epoch	29.44 s	21.71 s
Test accuracy	68.90%	70.43%
Macro precision	0.7249	0.7113
Macro recall	0.6379	0.6824
Macro F1	0.6597	0.6927
Calculated MACs	62.33 M	7.26 M

Model B improves test accuracy by 1.53 percentage points while reducing the parameter count by 98.51% and the Conv/Linear MAC count by approximately 88.35%. Its macro F1 also improves from 0.6597 to 0.6927. The main weakness in both custom models is H2, but Model B raises H2 recall from 0.2336 to 0.4387, reducing the severe bias toward H1 seen in Model A.

6 Task 5: Lightweight SOTA Fine-Tuning and Evaluation

The two selected lightweight pre-trained models are MobileNetV2 and ShuffleNetV2 1.0x. Both start from torchvision ImageNet weights, replace the 1000-class head with a five-class head, and are fully fine-tuned on the same DeFungi split for 20 epochs using Adam with learning rate 5×10^{-4} and weight decay 10^{-4}.

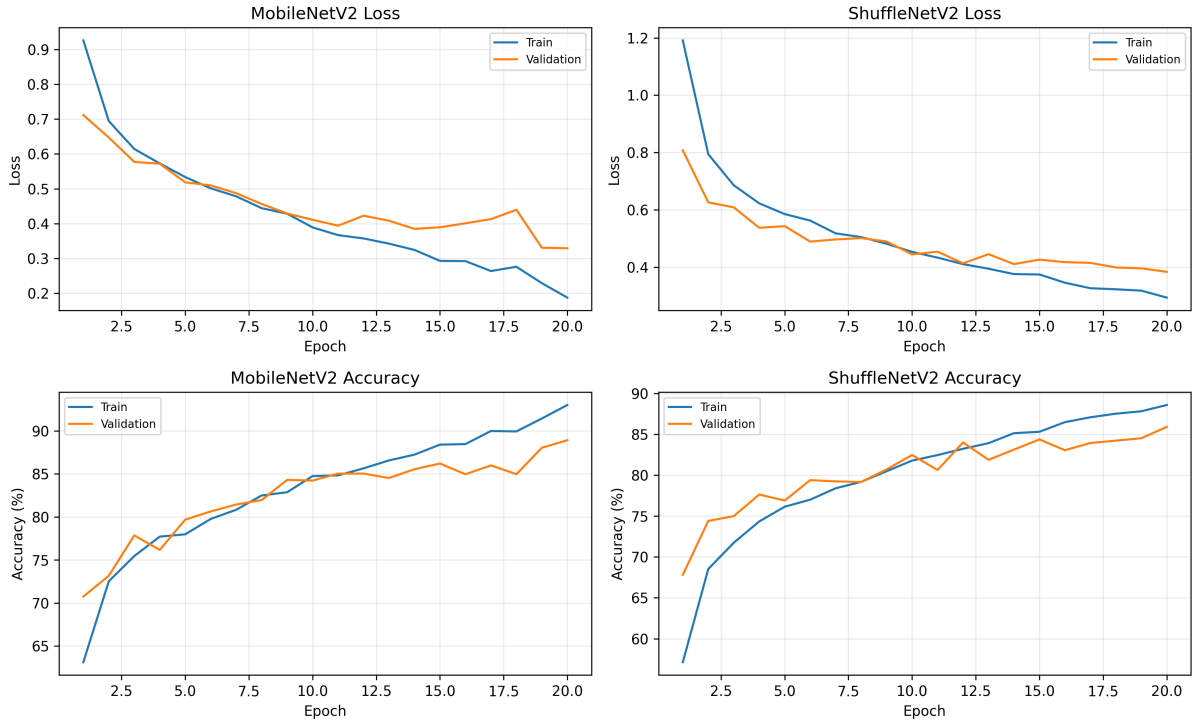
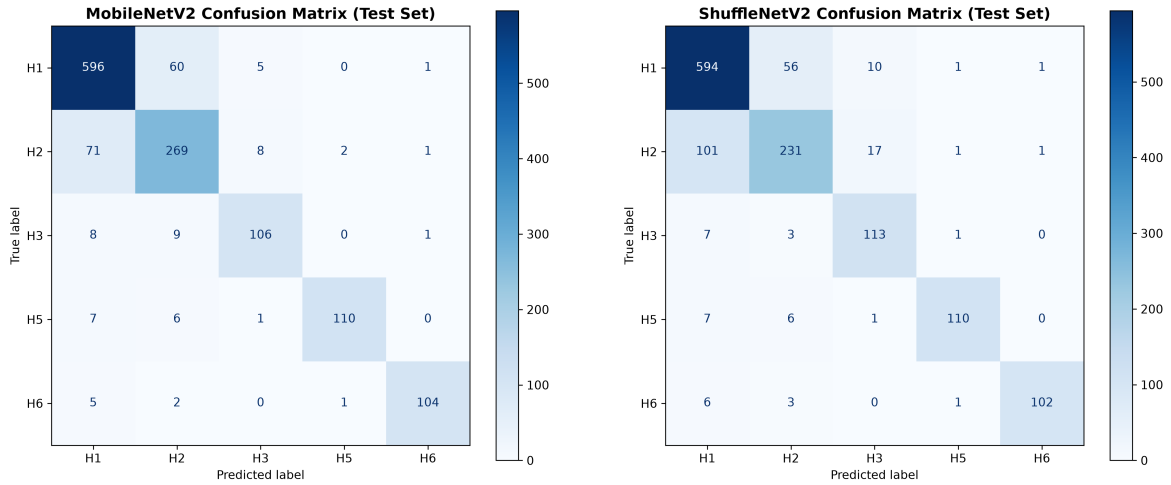


Figure 6: Training and validation curves for MobileNetV2 and ShuffleNetV2.



(a) MobileNetV2.

(b) ShuffleNetV2 1.0x.

Figure 7: Test-set confusion matrices for the two fine-tuned lightweight pre-trained models.

Table 3: Lightweight SOTA comparison.

Model	Params	Size	Test Acc.	Macro F1	Time/ep	MACs
MobileNetV2	2,230,277	8.740 MiB	**86.31%**	**0.8805**	40.90 s	24.45 M
ShuffleNetV2 1.0x	1,258,729	4.966 MiB	83.76%	0.8590	**20.59 s**	**11.75 M**

MobileNetV2 provides the strongest classification result, while ShuffleNetV2 provides a better

efficiency point among the two SOTA models. The improvement is especially visible on H2: Model B's H2 recall is 0.4387, ShuffleNetV2 reaches approximately 0.66, and MobileNetV2 approximately 0.77.

7 Task 6: Final Comparison and Trade-off Discussion

7.1 Accuracy

The main final comparison is Model B vs ShuffleNetV2 vs MobileNetV2.

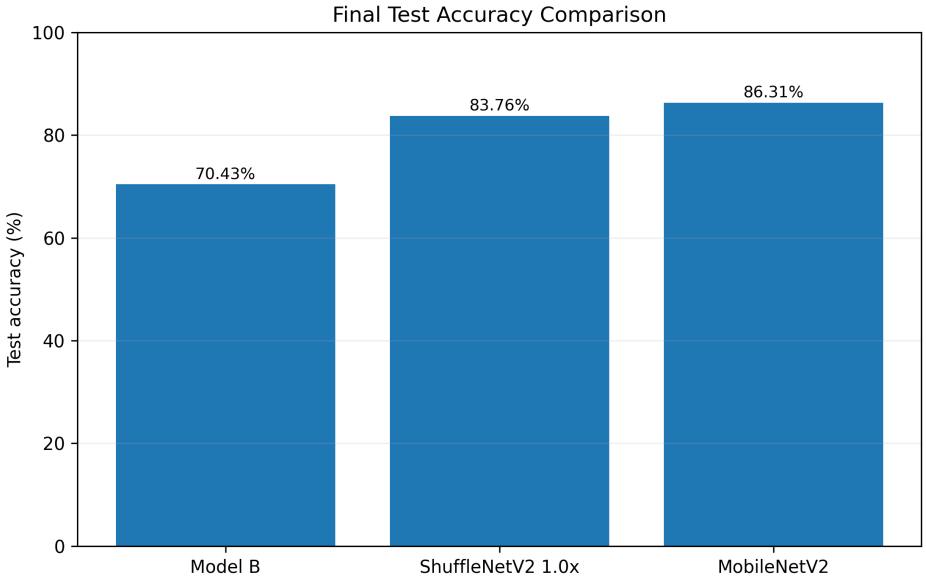
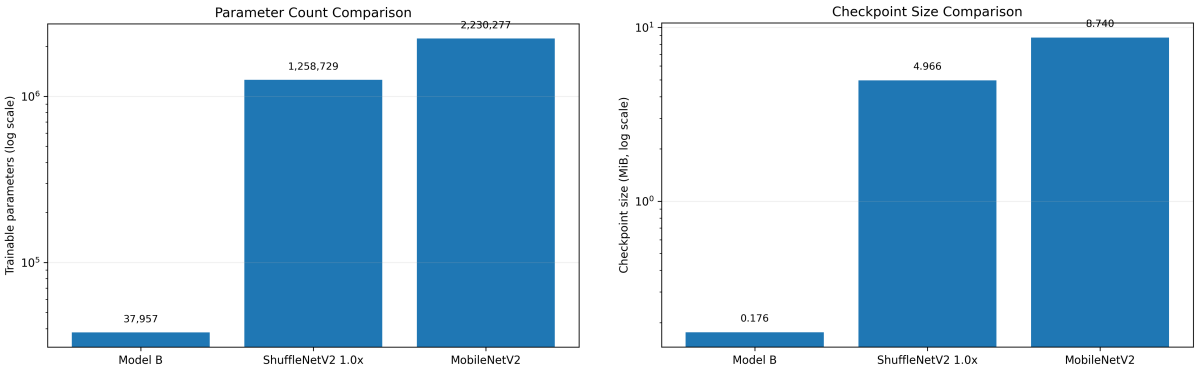


Figure 8: Test accuracy comparison for the resource-constrained custom model and the two fine-tuned SOTA models.

MobileNetV2 improves on Model B by **15.88 percentage points**; ShuffleNetV2 improves by **13.33 percentage points**. Transfer learning therefore provides a large performance advantage on this low-resolution microscopic dataset.

7.2 Memory footprint



(a) Trainable parameters.

(b) Checkpoint file size.

Figure 9: Memory-related model comparisons. Log scales are used because of the large size ratios.

Relative to Model B, ShuffleNetV2 has approximately **33.16**× as many parameters and a **28.23**× larger checkpoint. MobileNetV2 has approximately **58.76**× as many parameters and a **49.68**× larger checkpoint.

Checkpoint size is not equivalent to deployment RAM. Runtime activation buffers, framework overhead, operator implementations, quantization, and allocator behavior also affect peak memory. Consequently, this report does not claim that Model B is guaranteed to fit a particular microcontroller from checkpoint size alone.

7.3 Computational cost

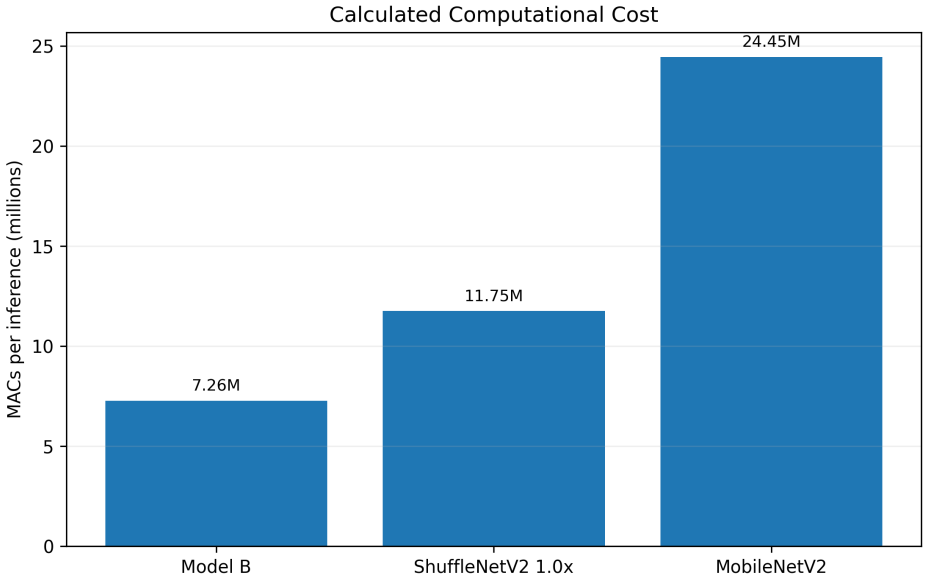
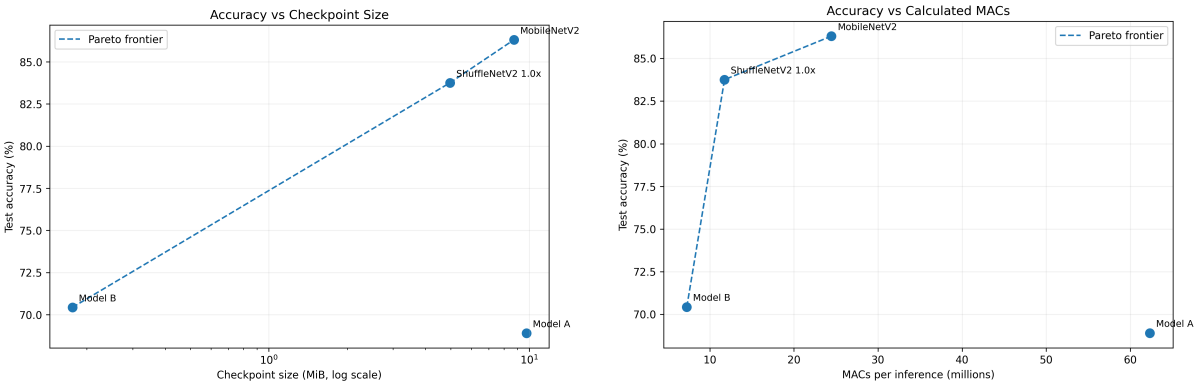


Figure 10: Calculated Conv/Linear MACs for a $3 \times 64 \times 64$ input.

Model B requires approximately 7.26 M MACs, compared with 11.75 M for ShuffleNetV2 and 24.45 M for MobileNetV2. ShuffleNet therefore uses approximately **1.62**× the MACs of Model B, while MobileNetV2 uses approximately **3.37**×. No controlled device-level inference latency was measured, so training time per epoch is not presented as inference latency.

7.4 Accuracy-efficiency frontier



(a) Accuracy vs checkpoint size.

(b) Accuracy vs calculated MACs.

Figure 11: Accuracy-efficiency trade-offs. The dashed line marks the non-dominated Pareto frontier; Model A is dominated by Model B because Model B is both smaller and more accurate.

Model B, ShuffleNetV2, and MobileNetV2 occupy different non-dominated operating points. Model B minimizes resource demand. ShuffleNetV2 sacrifices some accuracy relative to MobileNetV2 but offers substantially lower parameters, storage, and MACs. MobileNetV2 maximizes accuracy at the highest resource cost among the three.

7.5 Advantages and limitations of Model B

Advantages:

- Strictly satisfies the sub-100k trainable-parameter requirement.
- Very small checkpoint and low calculated arithmetic cost.
- Simple depthwise-separable building blocks and global average pooling reduce dense-layer overhead.
- Does not depend on ImageNet pre-training.

Limitations:

- Lower test accuracy than both fine-tuned SOTA models.
- Lower representational capacity because of the aggressive parameter budget.
- Training from scratch must learn both low-level and task-specific features from the comparatively small DeFungi dataset.
- H2 remains the most difficult major class, with recall 0.4387 in the main Model-B run.

7.6 Advantages and limitations of pre-trained SOTA models

Advantages:

- ImageNet pre-training supplies reusable low-level and mid-level visual features.
- Both SOTA models substantially improve test accuracy and macro F1.
- MobileNetV2 gives the highest accuracy; ShuffleNetV2 offers a strong middle point between accuracy and resource cost.

Limitations:

- Much larger parameter and storage footprints than Model B.
- Higher calculated arithmetic cost.

- Pre-training creates a domain-transfer dependency: ImageNet features are useful, but the source domain differs from microscopic fungi.
- Actual edge-device feasibility still depends on peak activation memory, operator support, quantization, and measured latency.

7.7 Deployment recommendations

- **Strictest memory/compute budget:** choose Model B. It is the only custom sub-100k model and has the lowest MAC count and checkpoint size. Before claiming deployment on a specific MCU, perform quantization, peak-RAM profiling, and an on-device latency test.
- **Balanced accuracy and efficiency:** choose ShuffleNetV2 1.0x. It improves accuracy by 13.33 percentage points over Model B with substantially less storage and MAC cost than MobileNetV2.
- **Accuracy-first edge deployment:** choose MobileNetV2. It reaches 86.31% test accuracy and macro F1 0.8805, at the cost of the largest checkpoint and highest MAC count among the three final candidates.

8 Overall Conclusion

The experiments demonstrate that depthwise-separable design can reduce resource cost dramatically without necessarily reducing accuracy. Model B is 98.51% smaller in parameter count than Model A, requires about 7.26 M Conv/Linear MACs, and slightly improves both test accuracy and macro F1. However, transfer learning changes the accuracy-efficiency frontier: MobileNetV2 reaches 86.31% test accuracy and ShuffleNetV2 reaches 83.76%, versus 70.43% for Model B.

The correct engineering choice therefore depends on the deployment constraint. Model B is the strongest candidate when model capacity must remain extremely small; ShuffleNetV2 is the most attractive compromise when moderate resources are available; and MobileNetV2 is preferred when accuracy dominates the resource budget. These conclusions are based on verified checkpoint sizes, measured test metrics, and consistently calculated MAC counts. Device-specific latency and peak runtime RAM remain future deployment measurements rather than assumed results.

Reproducibility Notes

The corrected code includes repository-relative paths, explicit seed handling, the packaged split manifest, a dedicated Task-3 optimizer-comparison script, and a common computational profiling script. Existing valid Task-4 and Task-5 checkpoints were left unchanged.

References

[1] UCI Machine Learning Repository, *DeFungi*, Dataset ID 773. `https://archive.ics.uci.edu/dataset/773/defungi`.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *Proc. CVPR*, 2018.

[3] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," *Proc. ECCV*, 2018.

[4] PyTorch / Torchvision model implementations and pre-trained weights documentation, `https://pytorch.org/vision/stable/models.html`.